

BalotaChain on the Saksi Framework

Thesis Defense Reviewer

Part 1: Chapter I, Introduction | Part 2: Chapter II, Review of Related Literature | Part 3: Chapter III, Methodology | Part 4: Appendices | Quick Recall | Panel Q&A

Built from BalotaChain_Main_Paper_v3. Page references point to that build. Wording of every definition, criterion, and figure is kept as it appears in the manuscript; only the question framing and the reviewer notes are new.

PART 1: CHAPTER I, INTRODUCTION (pp. 7 to 15)

The two properties (the whole thesis in one line)

- **Integrity:** every vote is counted exactly as it was cast.
- **Privacy:** no party can link a voter to the contents of their ballot.

Traditional systems get both through procedural safeguards and institutional trust. Cryptographic voting replaces that procedural trust with mathematical guarantees. Blockchain adds a tamper-evident, decentralized record. A permissioned blockchain adds governance and access control over who maintains that record.

Load-bearing sentence, p. 8: blockchain by itself provides only an append-only, replicated ledger; the integrity, privacy, and verifiability properties come from the cryptographic protocol layered on it, not from the ledger alone. Expect a panelist to ask why blockchain is needed at all. This sentence is your answer.

Statement of the Problem (p. 8)

Empirical evidence on how a cryptographic voting framework on a permissioned blockchain would perform in Philippine elections, with respect to integrity, privacy, security, and performance, remains limited and design-specific, and findings obtained under other designs may differ under the integrated design studied here.

The 3 research questions

RQ	Question	Measures
1	To what extent does the proposed framework operate correctly when evaluated as an election system at multiple scales, as measured by six criteria?	(a) announced tally equals ground truth; (b) invalid ballots rejected; (c) duplicate votes rejected; (d) all accepted ballots included; (e) result independently verifiable from the public record; (f) all cryptographic proofs verify
2	What integrity, privacy, and security properties does the proposed system demonstrate under specified threat conditions?	Unlinkability, ballot secrecy, and resistance to each defined attack scenario
3	What performance characteristics does the proposed system show across election sizes representing selected Philippine electoral scales, and how do these compare to existing cryptographic voting systems on the same blockchain platform?	Latency, throughput, threshold-decryption time, resource consumption; cross-study comparison with Galal et al. [18]

RQ1 is the one that was revised after the adviser flagged operate correctly as too broad. The six criteria (a) to (f) are the fix. Be able to recite all six.

General objective and the 3 specific objectives (p. 9)

General: generate empirical data on how a cryptographic voting framework on a permissioned blockchain performs in Philippine elections with respect to integrity, privacy, security, and performance, by designing, implementing, and evaluating BalotaChain on the Saksi Framework.

- **1. Implement** Saksi as an integrated election system (cryptographic core, permissioned blockchain layer, voter app, trustee app, open-source verifier) and verify correct end-to-end operation across scales with synthetic populations.

- **2. Evaluate** integrity, privacy, and security by implementing concrete attacks for each adversary class and demonstrating resistance under the stated threat model while preserving voter-ballot unlinkability and end-to-end verifiability.
- **3. Characterize** performance across scales and ballot complexities on latency, throughput, threshold-decryption time, and resource consumption, compared against an existing system on the same platform.

Objective 1 maps to RQ1, Objective 2 to RQ2, Objective 3 to RQ3. Table 3.6 on p. 46 is the formal alignment matrix. If asked how the objectives connect to the evaluation, point there.

Scope (p. 10)

- **Two axes:** election scale (number of voters) and ballot complexity (single-position vs multi-position).
- **Scale range:** 1,000 (max clustered precinct under COMELEC rules) to 1,000,000 (largest measured tier), then two capstone tiers: 1,921,917 (three Zamboanga Peninsula provinces) and 3,524,078 (full ZAMBASULTA).
- **Capstone status:** committed, not conditional. Each outcome, completion or a measured scaling limit, is reported as a primary finding.
- **Ballot model:** equal number of candidates per position. Single-winner positions decided by plurality. Multi-seat positions (senatorial) decided by single non-transferable vote: leading candidates fill the seats, each voter still selects exactly one, so no additional cryptography.
- **Out of scope:** ballots marking more than one candidate for a position, and candidate lists of differing length, because they need a well-formedness proof over a selection limit greater than one.
- **Why single-position exists:** to compare directly with the baseline on the same platform so the comparison reflects cryptographic design, not workload.
- **Why multi-position exists:** to simulate realistic Philippine ballot conditions on a three-position ballot.

Limitations (the 4 statements to memorize)

- Simulation with synthetic populations. No deployment in any real electoral exercise. No human participants, no personal information.
- Baseline data only. Not a deployment, not a security certification, not a feasibility study of a national election.
- Benchmarks run on the researchers' own hardware; absolute figures are specific to that environment.
- Raft ordering gives crash fault tolerance, not Byzantine fault tolerance, and the trustee threshold is console-enforced rather than counted at endorsement (both declared on p. 47 to 48, Experimental Design).

Significance: the 4 audiences (p. 11)

Audience	What they get
The field	Empirical baseline data that does not currently exist; replaces assumption with evidence
Researchers	A reusable, domain-independent cryptographic framework; an open-source verifier; a benchmark to measure against
Election administrators and policymakers	Concrete evidence on behavior and resource requirements
The broader public	Elections whose correctness can be independently verified rather than merely trusted

Definition of Terms: the ones most likely to be asked (pp. 11 to 15)

Term	Definition as used in the study
End-to-End Verifiability	A voter can confirm their vote was recorded as cast and counted as recorded, and any observer can confirm the announced tally is correct, without revealing how any individual voted.

Term	Definition as used in the study
Permissioned Blockchain	A distributed ledger whose participating nodes are known and authorized. Here, Hyperledger Fabric.
Threshold ElGamal Encryption	Decryption capability divided among several parties; needs a minimum number (threshold) to cooperate; no single party can decrypt alone.
Distributed Key Generation	Multiple parties jointly produce a public key while each holds only a share of the private key; the complete private key never exists in any single location.
Homomorphic Aggregation	Encrypted ballots combine into a single encrypted tally without decrypting any individual ballot; only the aggregate is ever decrypted.
Zero-Knowledge Proof	Proves a statement is true without revealing anything beyond its truth. Used for ballot well-formedness and honest decryption.
Blind-Signed Anonymous Credential	Authenticates a voter as eligible while preventing any party, including the issuer, from linking the credential to the ballot.
Bulletin Board	The public, append-only record of the election. Here, the Fabric ledger. Same artifact viewed as a cryptographic abstraction (bulletin board) and as a platform component (ledger).
Trustee	One of five independent parties holding a share of the election private key; any three may jointly decrypt the aggregate, no smaller group can.
Election Authority	Publishes the manifest and operates registration; holds no decryption capability.
Nullifier	A single-use value that marks a credential as having voted for a given position; prevents double voting while preserving anonymity.
Ballot Secrecy vs Unlinkability	Secrecy: the content of any individual ballot cannot be read by any party at any point. Unlinkability: no party can associate a voter with their ballot, even with full access to the public record.
Tally	Aggregate count per candidate. No individual ballot is decrypted for the tally, only the aggregated value.
Saksi Framework	The application-agnostic cryptographic library. The name is used only for the library, never for the election application.
BalotaChain	The election application built on Saksi: voter app, trustee console, chaincode, verifier.
Election System and Voting System	Used interchangeably for the complete deployed arrangement. Voting framework is not used except for Saksi.

Terminology consistency was adviser comment #30. If a panelist catches you saying BalotaChain framework or Saksi system, correct yourself immediately. Saksi is the library, BalotaChain is the application.

PART 2: CHAPTER II, REVIEW OF RELATED LITERATURE (pp. 16 to 29)

How the chapter is organized (the 5 parts, p. 16)

- 2.2 Cryptographic primitives underlying verifiable voting
- 2.3 End-to-end verifiable systems that combine those primitives
- 2.4 Blockchain-based systems that use a blockchain as the bulletin board
- 2.5 Formal security models that define verifiability and coercion resistance
- 2.6 Real deployments

Then 2.7 Synthesis and Research Gap, 2.8 Review Matrix (Table 2.2, twenty studies), 2.9 Conceptual Framework.

Philippine anchor: nationwide automated elections since 2010 using optical-scan technology, with repeated concerns about transparency and auditability [3].

2.2 The 4 primitives (pp. 17 to 18)

Primitive	Ref	Year	What it gives the system
ElGamal public-key encryption	[4]	1985	Security from the discrete-logarithm problem. Homomorphic: product of two ciphertexts decrypts to the sum of plaintexts. This is what lets encrypted ballots combine into an encrypted tally.
Pedersen verifiable secret sharing and commitments	[5]	1991	Non-interactive, information-theoretically secure. Distributes the decryption key so no individual holds it. Basis of distributed key generation.
Chaum-Pedersen proof of equality of discrete logarithms	[6]	1992	Lets a trustee prove a ciphertext was decrypted correctly with respect to a committed key share, without disclosing the share. Used at tally time.
Cramer, Damgard, Schoenmakers proofs of partial knowledge (disjunctive / OR proofs)	[7]	1994	Proves a ciphertext encrypts one of a set of valid values without revealing which. Used at ballot submission to prove well-formedness.

The recurring pattern: encrypt each vote, aggregate the ciphertexts homomorphically, prove well-formedness and correct decryption in zero knowledge, and decrypt only the final tally through a distributed set of trustees. Figure 2.1 shows the layered composition.

2.3 End-to-end verifiable systems (pp. 19 to 20)

System	Ref	Authors	Contribution	Limitation
Pret a Voter	[8]	Chaum, Ryan, Schneider	E2E verifiability on paper ballots via an encrypted receipt	Logistical overhead of secure ballot printing
Helios	[9]	Adida	First widely used web-based open-audit system; homomorphic ElGamal plus ZK proofs; ran binding elections	Centralized: one server runs the bulletin board, one trustee holds the key; no coercion protection
Civitas	[10]	Clarkson, Chong, Myers	First working coercion-resistant system; anonymous credentials let a coerced voter deceive the coercer	Heavy computational cost limits scalability
Selene	[12]	Ryan, Ronne, Iovino	Tracker numbers let a voter find their own vote in plaintext; deniable tracker assignment blunts coercion	Mitigation, not full resistance; tracker management

System	Ref	Authors	Contribution	Limitation
Belenios	[13]	Cortier, Gaudry, Glondou	Digital-signature authentication gives eligibility verifiability and blocks ballot stuffing	Voter-ballot link stays visible on the board; trusted registrar
ElectionGuard	[14]	Benaloh, Naehrig, Pereira, Wallach	Toolkit: threshold ElGamal, DKG, disjunctive proofs; ran US public elections	Bulletin-board substrate deferred to deployer; guardian independence listed as open problem

2.4 Blockchain-based systems (pp. 21 to 22)

System	Ref.	Platform	Key fact	Limitation
Open Vote Network	[15]	Ethereum (public)	First decentralized self-tallying internet voting protocol with maximum voter privacy; smart contract	Boardroom scale (40 voters on testnet); public-chain transaction cost
Diaz-Santiso and Fraga-Lamas	[16]	Hyperledger Fabric	Measured latency and load with Hyperledger Caliper	Limited cryptography; prototype
Pereira and colleagues	[17]	Custom chain (design)	Signatures plus homomorphic encryption plus ZKPs on BFT consensus	Conceptual design analysis; no measured implementation
Galal, El Reheem, Guirguis (THE BASELINE)	[18]	Hyperledger Fabric	AES-256 for ballot content, RSA-2048 for key protection; synthetic populations up to 50,000 voters; 1.07 s per vote, about 4.66 TPS	All decryption authority in a single election commission holding the RSA key; no threshold, no homomorphic tally
Surveys: Vladucu et al.; Jafar et al.	[1], [2]	N/A	Map the design space	Both keep pointing to scalability and squaring transparency with privacy

Reference [17] was mischaracterized in an earlier draft as a custom blockchain prototype. It is a conceptual design with no measured implementation. This was caught in the verification pass and corrected in Table 2.2 and Section 2.7. If asked whether every claim in the matrix was checked against its primary source, the answer is yes, and this is the example of what the check found.

2.5 Security models (p. 22)

- **Coercion resistance:** formalized by Juels, Catalano, and Jakobsson [11]; adversary can coerce voters or buy votes; their anonymous-credential construction influenced nearly every later coercion-resistant design.
- **Verifiability, unified:** Cortier, Galindo, Kusters, Muller, and Truderung [19], SoK 2016, distinguishes individual, universal, and eligibility verifiability within one formal framework.

The study does NOT claim coercion resistance, only unlinkability and end-to-end verifiability. If asked, say coercion resistance in the JCJ sense is outside the stated threat model and point to Table 3.9.

2.6 Electronic voting in practice (p. 23)

- **Estonia:** binding nationwide internet voting since 2005; Ehin, Solvak, Willemson, Vinkel [20] cover eleven elections 2005 to 2019; internet voting grew to nearly half of all votes cast.
- **Philippines:** Reyes [3] on the 2010 automated presidential election; automation changed how the process ran but left open questions about transparency and auditability.
- **Lesson:** electronic voting works in practice, but proprietary vendor systems and the lack of E2E cryptographic verifiability remain serious concerns.

2.7 Synthesis and the gap (pp. 23 to 25)

One trait shared by every approach: each solves only part of the problem.

Group	What it solved	What it left open
Primitives (2.2)	All the pieces for verifiable, private voting	Not full systems on their own
E2E systems (2.3)	Each adds something useful	Helios centralizes board and key; Belenios exposes voter-ballot link; ElectionGuard defers substrate and leaves trustee independence unsolved
Blockchain systems (2.4)	Decentralize the bulletin board	OVN is boardroom scale; Pereira is conceptual; Galal centralizes decryption and uses a scheme that cannot support homomorphic aggregation or threshold decryption
Formal models (2.5)	Definitions of verifiability and privacy	Build and measure no systems
Deployments (2.6)	Prove national-scale operation and adoption	Cryptographic verifiability outside their scope

Closing line to memorize: *Practice has thus run ahead of verifiable cryptography, and verifiable cryptography has run ahead of deployment-scale evidence.*

The gap, in one sentence: the absence of an integrated framework that combines threshold ElGamal encryption, distributed key generation across mutually independent trustees, zero-knowledge proofs of both ballot well-formedness and correct decryption, blind-signed anonymous credentials, and homomorphic aggregation, all deployed on a production-grade permissioned blockchain, and whose integrity, privacy, security, and performance have been characterized empirically at Philippine scales, up to the full ZAMBASULTA electorate of 3,524,078 registered voters.

The 7 properties of Figure 2.3 (p. 25) and Table 2.1

#	Property	Helios [9]	Belenios [13]	ElectionGuard [14]	OVN [15]	Galal [18]	Proposed
1	Threshold decryption	No	Partial	Yes	No	No	Yes
2	Distributed bulletin board	No	No	Partial	Yes	Yes	Yes
3	Ballot well-formedness proof	Yes	Yes	Yes	Partial	No	Yes
4	Cryptographic unlinkability	Partial	No	Partial	Yes	Partial	Yes
5	Homomorphic aggregation	Yes	Yes	Yes	Partial	No	Yes
6	Production-grade substrate	Partial	Partial	Partial	Partial	Yes	Yes
7	Philippine-scale evaluation	No	No	No	No	Partial	Yes

Table 2.1 row for the proposed system: Hyperledger Fabric | Threshold ElGamal | 3-of-5 trustees | End-to-end | Blind-signed credentials.

Adviser comment #6 warned about the claim that BalotaChain is the only system satisfying all seven. The safe phrasing is: among the systems reviewed, only the proposed framework satisfies all seven. Do not say no system in the world does.

2.9 Conceptual framework (p. 28, Figure 2.4)

- **Input:** validated synthetic voter populations with known per-position ground truth, across scale and ballot complexity, plus the threat and adversary conditions.
- **Process:** the framework operating as an E2E verifiable election: encryption, ZK proof generation, homomorphic aggregation, threshold decryption, verification on a permissioned blockchain.
- **Output and loop:** empirical characterization across four properties: accuracy (against ground truth), privacy (adversarial unlinkability), security (defined adversary classes), performance (quantitative); closed by a design

and evaluation feedback cycle. Adviser comment #8 asked for this to be research-specific rather than a generic IPO diagram.

PART 3: CHAPTER III, METHODOLOGY (pp. 30 to 59)

Research design: DSRM (p. 30, Table 3.1 on p. 33)

Design Science Research following the DSRM process model of Peffers and colleagues. Fits because the main output is an artifact. The work is applied and simulation-based.

DSRM phase	Application in this study	Where
1. Problem identification	Empirical evidence on E2E verifiable, privacy-preserving elections on a permissioned blockchain at PH scale is limited and design-specific	Ch I Background, Problem
2. Objectives of a solution	Saksi satisfying the seven properties; BalotaChain demonstrating correctness, verifiability, unlinkability, measured performance	Ch I Objectives; Fig 2.3
3. Design and development	Saksi: threshold ElGamal on ristretto255, Pedersen DKG 3-of-5, CDS proofs, Chaum-Pedersen proofs, blind-signed credentials, deterministic nullifiers. BalotaChain: Fabric with Raft, Go chaincode with three gates, homomorphic aggregation, Flutter voter app, Tauri trustee console, open verifier	Ch III Saksi Framework, Software Development
4. Demonstration	Full lifecycle on synthetic workloads from 1,000 to 3,524,078 voters, including a worked multi-position election	Ch III Experimental Design
5. Evaluation	Accuracy vs ground truth; rejection, unlinkability, attack resistance; latency, throughput, resources with percentiles; verifier reproduces tally; tampering trial; cross-study comparison	Ch III Testing and Evaluation; Ch IV
6. Communication	This manuscript plus separately versioned open-source Saksi and BalotaChain repositories	Whole manuscript

Epistemic status rule (p. 34): every capability statement is design intent (architecture intends it), implemented (realized in code), or demonstrated (established by experiment). No property is claimed beyond the strongest status it has attained.

This is your shield against overclaiming, adviser comment #31. If a panelist pushes you to say the system is secure, answer with the status ladder: designed for, implemented, demonstrated under the stated conditions and threat model.

The Saksi Framework (pp. 34 to 36)

- **Name:** Saksi is the Filipino word for witness.
- **Split:** Saksi is a reusable, independently versioned Rust library with no assumptions about elections. BalotaChain supplies the election workflow, user apps, and blockchain deployment. The split holds at repository level so the core can be tested and audited in isolation against published test vectors.
- **5 components:** Saksi cryptographic core (Rust); Fabric chaincode (Go); Flutter voter application; Tauri trustee console; open-source verifier.

The workflow, Figure 3.3 (p. 36): 7 stages in order

Stage	What happens	Primitive
1. Setup	Five trustees jointly generate the election key; no single party holds the whole private key; any three of five required to decrypt	Pedersen DKG
2. Registration	Eligible voters obtain credentials that prove eligibility without linking voter to ballot	Blind signatures

Stage	What happens	Primitive
3. Voting	Voter app encrypts the selection under the election key, attaches a proof of well-formedness and a single-use nullifier	ElGamal + CDS + nullifier
4. Submission	Chaincode independently verifies credential, nullifier, and proof. Only ballots passing all three gates are committed	The three gates
5. Aggregation	Accepted ballots combined into an encrypted total without decrypting any ballot	Homomorphic aggregation
6. Tally ceremony	At least three trustees produce partial decryptions, each with a proof of honest decryption; Lagrange interpolation recombines them	Chaum-Pedersen + threshold
7. Verification	The verifier recomputes the aggregate and re-checks every proof from the bulletin board	Open verifier

- **Beyond elections (design intent only):** healthcare population statistics from encrypted patient inputs; sealed-bid auctions and procurement. Explicitly not evaluated; no claim made about performance there.

Data collection (pp. 37 to 39)

Not collected from human voters. Synthetic populations generated by code, following Galal et al. [18] who used synthetic populations up to fifty thousand. Each record is one vote by one voter for one position. A multi-position voter produces one record per position sharing the voter_id but with its own nullifier and selection. Double voting is prevented per voter per position, not per voter overall.

Table 3.2: fields of a synthetic voter record

voter_id | position | credential | nullifier | selection | scale_group | ballot_complexity | ground_truth_tally

The 7 scale tiers and their real-world analogs (memorize the numbers)

Tier	Real-world analog	Source
1,000	Maximum clustered precinct	COMELEC Resolution No. 10965 [22]
10,000	Barangay cluster	
50,000	Municipality; also matches the baseline's measured population	[18]
483,000	Zamboanga City, 483,058 registered voters in 2025	[21]
1,000,000	Provincial-scale stress test, approaching Zamboanga del Sur at 1,196,013	[21]
1,921,917 (capstone)	Zamboanga del Norte 780,540 + Zamboanga del Sur excl. city 712,955 + Zamboanga Sibugay 428,422	2025 COMELEC statistics [21]
3,524,078 (capstone)	Full ZAMBASULTA: the above + Zamboanga City 483,058 + Isabela City 83,477 + Basilan 315,601 + Sulu 464,393 + Tawi-Tawi 255,632	2025 COMELEC statistics [21]

The staged ladder below the capstones is the engineering discipline that makes them reachable.

Data validation gate (p. 39): 4 checks before any data is used

- Every record complete and well-formed
- Identifiers and nullifiers unique
- Selections fall within the valid candidate set
- Aggregate ground-truth counts internally consistent

Failing data is regenerated or corrected. Purpose: results reflect how the system behaves, not flaws in malformed input.

Algorithms (pp. 39 to 42)

Shared facts for all four

- **Group:** ristretto255, a prime-order group abstraction over Curve25519; approximately 252-bit scalar field; scalars and group elements are 32-byte encodings.
- **Randomness:** private keys and nonces are uniformly random scalars from the OS CSPRNG via the Rust getrandom interface.
- **Non-interactivity:** all ZK proofs use the Fiat-Shamir transform over a domain-separated transcript hash.
- **Ciphertext:** $(c1, c2) = (g^r, g^m \cdot h^r)$, where g is the generator, h the election public key, m the encoded message, r the nonce.

Algorithm	Ref.	Key facts to be able to explain
1. Threshold ElGamal on ristretto255	[4]	Selection is a bit b , lifted into the group as g^b before encryption (lifted encoding). Component-wise product of all accepted ciphertexts gives $(g^{\sum r}, g^{T \cdot h^{\sum r}})$, T = votes for that selection. Trustees jointly remove the mask $h^{\sum r}$, leaving g^T . T recovered by bounded discrete-log search: linear scan at small tiers, Shanks baby-step giant-step [29] at 50,000 and above, bound = ballots accepted at that tier, so recovery is exact and terminates.
2. Pedersen DKG	[5]	Five trustees jointly generate the public key; each holds only a share. Three of five needed to decrypt; no single trustee and no group under three can. Removes the single point of compromise of single-authority designs.
3. Zero-knowledge proofs	[7], [6]	At submission: CDS disjunctive proof shows the ballot holds exactly one valid candidate without revealing which. At tally: Chaum-Pedersen proof lets each trustee prove its partial decryption was computed honestly from its committed share.
4. Blind-signed credentials		Registrar signs a blinded value; voter gets a valid credential; registrar never learns the value that identifies the ballot. Multi-position: one contest per position with its own encrypted accumulator; nullifier derived per credential and position by a domain-separated hash. Worked example: 3 positions, 5 voters, 15 records admitted, aggregation per contest, three tallies each summing to 5. Hot key at multi-million scale: every ballot read-modify-writes the same accumulator. Fix: per-precinct sub-accumulators folded into the contest total in one closing transaction. Correct because ciphertext addition is associative and commutative; the fold is on the board and re-checked by the verifier.

Adviser comment #11 was specifically about how the final count is recovered from the decrypted group element. The answer is the bounded discrete-log search. Be ready to say why it always terminates: the search bound equals the number of accepted ballots, and T cannot exceed that.

Software development and tools (pp. 42 to 44)

- **6 increments:** cryptographic core, election protocol, blockchain integration, client applications, testing, evaluation.
- **Validation ladder:** the complete protocol is validated end to end at 1, 10, 100, and 1,000 voters before any larger tier. A failed stress tier is a scaling-limit finding, not a failure of the study.
- **Traceability:** each repository documents version tags, dependency manifests, build instructions, test commands, deployment steps, so any result traces to the exact commit.
- **Figure 3.4:** architecture with trust boundaries: client side, Sakshi library, Fabric network with three chaincode gates and Raft ordering, public ledger, verifier path entirely outside every trust boundary.
- **Table 3.4 tools:** Rust, Go, Flutter/Dart, Tauri, Hyperledger Fabric, Hyperledger Caliper, Docker, Git/GitHub (two repositories), VS Code, Zotero. All free. Grand total 0.

- **Hardware:** Apple M4 for client-side cryptographic workload; separate desktop hosts the Fabric network. No new hardware.

Adviser comment #28: software cost of zero is distinct from total research cost. Appendix D puts the total research cost at PHP 12,900.

Testing and evaluation (pp. 45 to 47)

- **3 testing tiers before any experiment:** unit (primitives: encryption round-trips, proof soundness and completeness, nullifier determinism and uniqueness, share generation and reconstruction at and below threshold, credential issuance); integration (app to library, library to chaincode via Fabric gateway, three gates against the ledger, verifier against a populated board); system (full lifecycle at 1,000 voters). All three must pass first.

Table 3.5 evaluation matrix (p. 45)

Scale	Single-position	Multi-position
1,000 / 10,000 / 483,000	Evaluated	Evaluated
50,000	Evaluated, direct comparison with [18]	Evaluated
1,000,000	Evaluated, compared with the beyond-one-million projection of [18]	Stretch goal
1,921,917 and 3,524,078	Capstone attempt (committed)	Capstone attempt (committed)

Note the one asymmetry: multi-position at 1,000,000 is a stretch goal while both capstones are committed. If asked, the multi-position capstone runs are committed; the 1M multi-position rung is the one that may be skipped.

Table 3.6 alignment matrix (p. 46): what each RQ is measured by

RQ	System function	Scenario	Metric	Expected
1(a)	Homomorphic aggregation and 3-of-5 decryption	T1	$E = \sum T_i - G_i $	$E = 0$ at every tier
1(b, c)	Proof-verification and nullifier gates	T4, T5	Rejection rate	All invalid and duplicate rejected
1(d, e, f)	Independent verifier on the public record	T1, T7	Ballots counted; proofs verified	All counted; all verify
2 privacy	Credentials, nullifiers, encrypted ballots	T5	Adversary linkage success rate	No better than anonymity-set baseline
2 security	Three gates and the trustee threshold	T3 to T6	Outcome per attack	Resistance demonstrated
3 latency	Submission and endorsement path	T1, T2, T8	p50, p95, p99	Reported per tier
3 throughput	Ordering and commit path	T2, T8	TPS	Per tier, with scaling limit
3 resources	Peer, orderer, client consumption	T2, T3, T8	CPU, memory, disk, network; success rate	Per tier

Experimental design (pp. 47 to 50)

- **Metric definitions follow:** the Hyperledger Performance and Scale Working Group white paper [23], operationalized by Caliper: send rate, throughput as committed TPS, latency as percentiles not averages, success rate, resource consumption.

Table 3.7: the 7 experiments

Experiment	Pass criterion or output
Accuracy	Tally error $E = 0$ at every tier
Integrity	100 percent rejection of invalid submissions (T4, T5), verifiable from the board
Reliability	Repeat each configuration at least three times; restart a node under load (T3); 100 percent commit success, zero ballot loss
Response time	p50, p95, p99 per tier under normal (T1) and peak (T8) load; characterization, no fixed threshold
Scalability	Identical workload across all tiers; degradation documented per tier
Throughput	Caliper load at increasing send rates until saturation; committed TPS compared with [18], [26]
Performance log analysis	Caliper reports, peer and orderer logs, chaincode events consistent with reported metrics

Table 3.8: the 8 scenarios (nine-column schema)

ID	Scenario	Action or attack	Expected behavior	Property
T1	Normal voting operations	Steady submission through the complete lifecycle	All valid ballots accepted; $E = 0$; latency recorded	Integrity; accuracy
T2	High-volume transactions	Sustained load approaching saturation	Throughput plateau documented; no accepted ballot lost	Availability; performance
T3	Network interruption and recovery	A peer or orderer stopped mid-run and restarted	Ledger consistent; accepted ballots preserved	Reliability
T4	Vote or return tampering	Malformed ballots, altered ciphertexts, manipulated manifests; modification of committed records	All rejected; records immutable; verifier detects modification	Integrity
T5	Unauthorized access	Invalid or expired credentials, reused nullifiers, duplicates	100 percent rejection, each logged with its violated gate	Eligibility; uniqueness
T6	Trustee validation and approval	Decryption with two shares, then three, including one invalid proof	Sub-threshold fails; invalid proof detected; three valid shares decrypt and sign	Secrecy; threshold integrity
T7	Concurrent voting	Parallel ballots across positions while interim aggregates are read	Per-position nullifier holds; every ballot counted exactly once	Uniqueness; accuracy
T8	Peak election conditions	Worst-case arrival burst sized from the tier's real-world analog	Run completes, or bottleneck documented as a scaling-limit finding	Availability; scalability

The nine columns: ID, scenario, preconditions, test input, action or attack, expected behavior, actual result and verdict, evidence or log reference, related security property. Adversary classes map onto T4, T5, and T6.

The two declared limitations (p. 47 to 48)

- Raft ordering provides crash fault tolerance, not Byzantine fault tolerance. Byzantine behavior is handled at the protocol layer through proof verification and the trustee threshold, not at the ordering layer.
- The trustee threshold is enforced by the election console and re-proved by the verifier; the chaincode validates each partial decryption but does not count them before accepting a published tally. Endorsement-time enforcement is a subsequent chaincode revision.

Accuracy (p. 50)

- **Formula:** $E = \sum |T_i - G_i|$ over all candidates, T_i the decrypted count, G_i the seeded ground truth. Accuracy confirmed when $E = 0$.
- **External standard:** cryptographic operations validated against published test vectors from the ElectionGuard reference toolkit [14].
- **Two verification experiments:** (1) the verifier on a separate machine, given only the public record, must reproduce the announced tally while accepting every proof; (2) one public record is deliberately modified and the verifier must detect it.
- **Why (2) matters:** detection demonstrates software independence in the sense of Rivest [28]: an undetected change in software or data cannot cause an undetectable change in the outcome.

Privacy (p. 51): 3 checks, all categorical

Check	What the adversary or test does	Privacy holds if
1. Unlinkability	Adversary holds the complete public bulletin board (encrypted ballots with proofs, nullifiers, aggregate, partial decryptions with proofs, registration list) and must output a voter-ballot pairing	Success rate does not exceed the reciprocal of the anonymity-set size (all voters at that tier), per Pfitzmann and Hansen [27]
2. Ballot secrecy	Inspection of the tally path	No individual ballot is ever decrypted, only the aggregate
3. Threshold	Decryption attempted with fewer than three shares	The attempt fails

Adviser comment #15 asked for privacy to be an actual experiment. The unlinkability check is now a testable hypothesis with a defined adversary, defined inputs, and a defined success criterion (better than 1/N).

Security (pp. 51 to 53): 2 levels**Protocol level: 5 adversary classes, one automated deterministic test each**

Malicious voter (double-vote or malformed ballot); sub-threshold malicious trustees (corrupt decryption); malicious administrator (tamper with manifest); malicious bulletin-board node (drop or reorder ballots); network adversary (interception or replay).

The 11 negative test cases, each rejected by the specific gate that guards it

Malformed ciphertext; invalid or substituted validity proof; invalid or expired credential; reused nullifier; duplicate vote for the same position; altered transaction; altered manifest; decryption with fewer than three shares; incorrect trustee decryption proof; unauthorized identity; corrupted ledger data.

Wording rule (adviser comment #14): a rejected attack demonstrates resistance to that defined scenario under the stated threat model; it is not read as a general claim that the system is secure.

Implementation level: 4 checks

- Input validation testing: malformed, boundary, oversized inputs at every external entry point
- Static application security testing: Rust analyzers for the core, Go analyzers for the chaincode
- Dependency and supply-chain scanning against published advisory databases
- Rust removes memory-safety vulnerability classes by language design

Threat model, Table 3.9 (p. 53): 6 adversaries

Adversary	Capability	Attack	Defense	Evidence
Malicious voter	Valid credential; own submissions	Double vote; malformed ballot	Nullifier gate; CDS proof gate	Rejection logged
Compromised client	Controls one voter's device	Substituted proof; altered ciphertext; credential misuse	Proof gate; credential gate; client holds no key material	Rejection; no key exposure

Adversary	Capability	Attack	Defense	Evidence
Malicious trustees (up to two)	Shares below threshold	Corrupt or withhold partials; sub-threshold decryption	3-of-5 threshold; Chaum-Pedersen proof per partial	Invalid proof detected; decryption impossible
Network adversary	Observes and replays traffic	Interception; replay	TLS; nullifier uniqueness; ciphertexts reveal nothing	Replay rejected; no plaintext
Ledger administrator	Operates peers or orderers	Drop, reorder, tamper	Append-only replicated ledger; independent verifier	Tampering detected by verifier
External attacker	No credentials; network only	Unauthorized submission; forged identity	Credential gate; membership service	Rejection logged

Why up to two trustees: three of five is the threshold, so the largest malicious set that stays below it is two. If three colluded, the threshold is met and secrecy of the aggregate is lost by design; the model assumes at least three honest trustees.

Legal mapping, Table 3.10 (p. 54)

Statute	Sections cited	Used for
Omnibus Election Code, BP 881	Sec. 261 election offenses (illegal voting); 261(j) falsification; Sec. 264 penalties	Double voting; malformed ballot; manifest manipulation
RA 8436 as amended by RA 9369 (automated election law)	Sec. 35 AES integrity; 35(a) tampering with the AES; Sec. 22 / 19 election return as official basis; Sec. 30 authentication via RA 8792	Trustee corruption, admin manipulation, node tampering, network access; election return verification
RA 10173, Data Privacy Act of 2012	Sec. 25 unauthorized processing; 28 unauthorized purposes; 29 unauthorized access or breach	Privacy adversary; network adversary
RA 8792, Electronic Commerce Act	Authentication procedures	Digital signature on election returns
National Press Club of the Philippines v. COMELEC	G.R. No. 259354, June 13, 2023	Supreme Court affirmed machine-generated digital signatures satisfy the requirement; signing mechanism is COMELEC's discretion

Disclaimer you must say verbatim if asked: these statutory references provide context for the simulated threat scenarios only; as the researchers are computer science students, no claim of legal validity, compliance, or certification is made or implied. Because the data are synthetic, the Data Privacy Act entries denote the conduct the property would prevent in a real deployment, not an offense arising from the simulation.

Election return verification and approval (p. 55)

The final decrypted tally is the system's analogue of an election return. Each participating trustee digitally signs the tally alongside its Chaum-Pedersen proof; approved only when at least three of five have signed. Signed tally, proofs, and signatures are committed to the board through consensus. This converts the single-machine digital signature of current practice into a threshold, publicly verifiable, consensus-anchored approval, compatible with the statutory framework. The system strengthens the existing mechanism rather than replacing it.

Audit trails, Table 3.11 (pp. 55 to 57)

The verifier's 10 checks, from the public record alone

(1) validity of every recorded transaction; (2) validity of each ballot's credential; (3) uniqueness of every nullifier; (4) well-formedness of every ciphertext; (5) the disjunctive validity proof of every ballot; (6) correctness of the

homomorphic aggregate; (7) each trustee's Chaum-Pedersen proof; (8) at least three of five trustees contributed; (9) announced tally equals the decrypted aggregate; (10) append-only consistency of the ledger.

Table 3.11: 9 stored items

On-chain, public, permanent	Off-chain, private, research period
Encrypted ballot; ballot validity proof; nullifier; credential token; partial decryptions and proofs; signed final tally; election manifest	Operational logs (gate outcomes, ceremonies, node events, benchmark reports); synthetic voter registry (eligibility list and source of ground truth)

- **Why no linkage:** no published field contains a voter identity; nullifiers and credentials are one-way; ciphertexts are semantically secure; the registry is synthetic, off-chain, private.
- **VVSG 2.0 [25]:** the ledger provides auditability, the verification procedure provides the evidence. An immutable record alone does not establish a correct outcome; the audit procedure over that record does.

Performance (pp. 57 to 59)

Item	Value
Baseline [18], same platform	About 1.07 s per vote, about 4.66 TPS at 50,000 voters. Different hardware, so cross-study comparison, not a controlled experiment.
Upper bracket [24]	Fabric 2.5 on commodity servers: about 3,000 TPS with one million assets in world state
Lower bracket [26]	Closest prior ElGamal homomorphic tally on Fabric: about 40 committed TPS. Proposed system expected between the brackets.
Repetitions	At least 10 measured runs after 2 discarded warm-ups. Capstone tiers: 3 measured after 1 warm-up, a standard accommodation for long-running benchmarks.
Reported statistics	Minimum, median, mean, p95, p99, standard deviation; plus CPU, memory, disk, network of peers, orderers, clients
Stages timed separately	Proof generation, proof verification, aggregation, threshold decryption
Scaling-limit rule	Sustained throughput below the arrival rate implied by a 10-hour voting window is a scaling limit of the evaluated configuration, not a failure of the design: about 28 ballots/s at 1M; about 54 at 1,921,917; about 98 at 3,524,078
Failed runs	Excluded from throughput, reported separately as a failure rate

Table 3.12 benchmarking environment

AMD Ryzen 7 5700G, 8 cores, 16 threads; 32 GB DDR4; NVMe SSD; LevelDB state database with history off; approximately 30 to 50 GB ledger per million single-position ballots, 150 GB provisioned for the full-ZAMBASULTA multi-position run; Fabric 2.5 with Raft; OS, container runtime, topology, toolchain versions, and container limits recorded at execution.

The 28 / 54 / 98 figures: divide voters by 36,000 seconds. $1,000,000 / 36,000 = 27.8$. $1,921,917 / 36,000 = 53.4$. $3,524,078 / 36,000 = 97.9$. Know the arithmetic, a panelist may ask where 98 comes from.

Ethical considerations (p. 59): the 5 commitments

- No real voter data collected at any stage
- No actual election conducted
- No real election credentials issued or used
- All simulated attacks executed only within the researchers' controlled environment
- No external or production election infrastructure targeted or probed

Also: only publicly available, openly licensed software; Sakshi and the verifier released open-source; any later phase with real participants would seek institutional review and Data Privacy Act compliance first.

PART 4: APPENDICES (pp. 63 to 74)

Appendix A: Data Collection (p. 63)

- **Table A.1:** ground-truth input before encryption. Wide format: one row per voter, one column per position. Stage-4 DSRM artifact, exported as ground-truth-ballots.csv with per-candidate totals in ground-truth-summary.csv.
- **Table A.2:** record after simulated registration. Stage 5. Credential repeats across a voter's rows; nullifier differs per row.
- **Generation parameters:** scales 1,000 to 3,524,078; three positions with President and VP single-winner and Senator multi-seat by SNTV; one record per position; ristretto255; fixed K candidates per position; selection distribution uniform, skewed, and realistic profiles; no random state, each selection is a pure arithmetic function of voter index, position index, candidate count, and profile; integer arithmetic so identical across architectures; fully synthetic.
- **Why the realistic profile exists:** uniform and skewed alone produce tied outcomes at every scale; the realistic profile applies the skewed rule to most of the electorate and apportions a reserved share down the candidate ranks so every position yields a decidable winner.
- **Sample ground truth, 500 voters single-position:** CAND_PRES_01 142, _02 119, _03 131, _04 108. Total 500.

Appendix B: Software Design / Prototype (p. 65)

Screens of the voter application, the public bulletin board, and the trustee console.

Appendix C: Evaluation Tool (p. 70)

- **Performance form:** 14 configurations, SP-1K to SP-3.5M and MP-1K to MP-3.5M; columns latency per ballot, TPS, threshold-decrypt time, resources (peak CPU and memory).
- **Accuracy form:** configuration, position, candidate, decrypted count, ground-truth count, absolute difference; E is the sum.
- **Privacy form:** unlinkability, ballot secrecy, sub-threshold resistance; result and notes.
- **Security form:** five protocol rows (one per adversary class) and four implementation rows.

Appendix D: Research Budget (p. 73)

Software cost zero. Estimated research cost PHP 12,900: electricity 2,500; internet 2,400; transportation 1,000; printing and binding 3,000; cloud 0; equipment depreciation 4,000.

QUICK RECALL: LIKELY ASSESSMENT POINTS

Numbers worth memorizing

Number	What it is
2	Properties in tension: integrity and privacy. Axes of evaluation: scale and ballot complexity. Verification experiments. Security levels. Declared limitations. Repositories.
3	Research questions. Objectives. Chaincode gates (credential, nullifier, proof). Trustees needed to decrypt. Positions on the multi-position ballot. Testing tiers. Privacy checks. Capstone repetitions. Verifiability notions (individual, universal, eligibility).
4	Cryptographic primitives. Data-validation checks. Properties in the output (accuracy, privacy, security, performance). Implementation-level security checks. Audiences in Significance. Algorithms.
5	Trustees. Components of BalotaChain. Adversary classes. Ethical commitments. Parts of the literature review. Scale tiers below the capstones.
6	DSRM phases. Development increments. RQ1 criteria (a) to (f). Adversaries in Table 3.9.
7	Properties in Figure 2.3. Scale tiers in total. Experiments in Table 3.7. Workflow stages in Figure 3.3.
8	Testing scenarios T1 to T8. Fields in Table 3.2.
9	Columns in Table 3.8. Stored items in Table 3.11.
10	Verifier checks. Measured repetitions per configuration (after 2 warm-ups).
11	Negative test cases in the security catalog. Estonian elections in [20].
20	Studies in the review matrix, Table 2.2. Reference numbers cited in the chapter body ([1] to [20]); [21] to [31] are supporting sources.
3-of-5	The trustee threshold.
252-bit / 32 bytes	ristretto255 scalar field size / encoding size.
1.07 s / 4.66 TPS / 50,000	The Galal et al. [18] baseline.
40 / 3,000 TPS	Lower [26] and upper [24] brackets.
28 / 54 / 98 ballots per second	Scaling-limit thresholds at 1M / 1.92M / 3.52M over a 10-hour window.
483,058 / 1,196,013 / 1,921,917 / 3,524,078	Zamboanga City / Zamboanga del Sur / three ZamPen provinces / full ZAMBASULTA, 2025 COMELEC.
30 to 50 GB / 150 GB	Ledger per million single-position ballots / provisioned for the full multi-position run.
PHP 12,900	Estimated total research cost. Software cost is zero.

Easy-to-confuse pairs

Pair	The distinction
Saksi vs BalotaChain	Saksi is the reusable, application-agnostic Rust library. BalotaChain is the election application built on it. Never say Saksi system or BalotaChain framework.

Pair	The distinction
Bulletin board vs blockchain / ledger	Same artifact. Bulletin board is the cryptographic abstraction; ledger is the Fabric platform component.
Ballot secrecy vs unlinkability	Secrecy: no one can read the ballot content. Unlinkability: no one can tie a voter to a ballot, even with the full public record.
CDS proof vs Chaum-Pedersen proof	CDS (disjunctive) at submission proves the ballot encodes exactly one valid candidate. Chaum-Pedersen at tally proves a partial decryption was honest.
Credential vs nullifier	Credential proves eligibility, issued once per voter, blind-signed. Nullifier is a single-use marker per credential per position that prevents double voting.
Accuracy vs integrity	Accuracy is the metric ($E = 0$). Integrity is the property it demonstrates. Table 3.7 has separate Accuracy and Integrity experiments: Accuracy compares the tally, Integrity confirms invalid ballots cannot enter.
Resistance demonstrated vs system is secure	Only the first is claimed. A rejected attack demonstrates resistance to that scenario under the stated threat model.
Design intent vs implemented vs demonstrated	The epistemic status ladder. Every claim carries its strongest attained status and no more.
Crash fault tolerance vs Byzantine fault tolerance	Raft gives CFT. Byzantine behavior is handled at the protocol layer (proofs, threshold), not the ordering layer.
Single-position vs multi-position	Single-position exists for a like-for-like baseline comparison. Multi-position exists for Philippine realism and is reported on its own.
Stretch goal vs capstone attempt	1M multi-position is a stretch goal. Both capstones are committed attempts with either outcome reported as a finding.
Controlled experiment vs cross-study comparison	Baseline ran on different hardware, so the comparison with [18] is cross-study, not controlled.
Plurality vs single non-transferable vote	Single-winner positions use plurality. Multi-seat positions use SNTV: leading N candidates take the seats, each voter still selects one. Seat count changes how the result is read, not how the ballot is proven.
Auditability vs evidence (VVSG 2.0)	The ledger provides auditability. The verification procedure provides the evidence.
Individual vs universal vs eligibility verifiability	Individual: a voter checks their own vote. Universal: anyone checks the tally. Eligibility: only eligible voters voted, once. From Cortier et al. [19].
Threshold at console vs at endorsement	Currently the console enforces 3-of-5 and the verifier re-proves it; the chaincode validates each partial but does not count them. Endorsement-time counting is the next chaincode revision.

One-line answers to expect

Question	Answer
What is the title?	BalotaChain on the Saksi Framework: End-to-End Verifiable Elections on Permissioned Blockchain Using Synthetic Election Data.
What does Saksi mean?	Witness, in Filipino.
What is the gap?	No integrated framework combining threshold ElGamal, DKG across independent trustees, ZK proofs of well-formedness and correct decryption, blind-signed credentials, and homomorphic aggregation on a production-grade permissioned blockchain, characterized empirically at Philippine scales.

Question	Answer
What is the research design?	Design Science Research following the DSRM process model of Peffers and colleagues; applied and simulation-based.
Why synthetic data?	The study characterizes a system's cryptographic and performance properties, so ground truth must be known; synthetic data gives that, removes human-subject recruitment and privacy exposure, and still exercises every operation. Galal et al. did the same.
Why Hyperledger Fabric?	Permissioned: participants are known and authorized; access control, throughput, suitability for elections among known authorities; and it is the same platform as the baseline, so the comparison isolates cryptographic design.
Why ristretto255?	A prime-order group abstraction over Curve25519 that avoids cofactor issues; approximately 252-bit scalars, 32-byte encodings.
Why 3-of-5?	No single trustee and no two can decrypt; up to two can be malicious or offline and the election still completes. It is the threshold ElectionGuard-style designs use and it is what Table 3.9 models.
How is the final count recovered?	Trustees remove the mask from the aggregate leaving g^T ; T is found by a bounded discrete-log search, linear at small tiers and baby-step giant-step from 50,000 up, bounded by the accepted ballot count so it always terminates exactly.
What prevents double voting?	The nullifier: a single-use value derived per credential and position by a domain-separated hash. The chaincode rejects a reused nullifier.
What prevents an invalid ballot?	The CDS disjunctive proof at the proof gate; a ballot must prove it encodes exactly one valid candidate.
What prevents a dishonest trustee?	The Chaum-Pedersen proof on each partial decryption; an invalid proof is detected and the partial is rejected.
What does the verifier need?	Only the public election record. It runs entirely outside every trust boundary and performs ten checks.
How is privacy measured?	As a testable hypothesis: an adversary with the full public record cannot link a voter to a ballot with probability better than $1/N$ over the anonymity set.
What is E?	The tally error, the sum over candidates of the absolute difference between decrypted and seeded counts. Pass is $E = 0$.
What is the baseline?	Galal, El Reheem, and Guirguis, Journal of Cloud Computing, 2026, on Hyperledger Fabric: AES-256 plus RSA-2048, 50,000 voters, about 1.07 s per vote and about 4.66 TPS, single election commission holds the RSA key.
What happens if the capstone fails?	The measured scaling limit and the observed bottleneck are reported as a primary finding. A failed stress tier is a finding, not a failure of the study.
Is the system secure?	It demonstrates resistance to the defined attack scenarios under the stated threat model. That is the only claim made.
What is not claimed?	Deployment, security certification, feasibility of a national election, coercion resistance, legal compliance, performance outside the benchmark environment, suitability outside elections.
What is the closing line of the review?	Practice has run ahead of verifiable cryptography, and verifiable cryptography has run ahead of deployment-scale evidence.

PANEL Q&A: HARD QUESTIONS AND WHERE THE ANSWER LIVES

These are the questions most likely to be asked given the adviser's 31 comments, the title-defense panel's ZAMBASULTA request, and what the manuscript leaves open. Each answer is drawn from the manuscript; the note says how much room you have.

Likely question	Answer from the manuscript	Where
Why do you need a blockchain at all? Couldn't a signed append-only log do this?	The blockchain is the bulletin board: a replicated, append-only record with access control and consensus over who may append. The security properties come from the cryptographic layer, not the ledger. What Fabric adds over a single signed log is that no single operator controls the record, and the ledger administrator is an explicit adversary in Table 3.9, defended by replication plus the independent verifier.	p. 8 (Background), p. 53 (Table 3.9), p. 57 (VVSG note)
Raft is not Byzantine fault tolerant. How can you claim integrity if the orderers can collude?	Stated as a declared limitation. Ordering-layer Byzantine behavior is not defended at the ordering layer. Integrity rests on the protocol layer: a colluding orderer can drop or reorder but cannot forge a valid ballot (proof gate), reuse a nullifier, or produce a valid partial decryption. The verifier detects omission and tampering from the public record.	p. 47 to 48
Your threshold is enforced by the console, not the chaincode. So a compromised console could publish a two-share tally?	Declared limitation. The chaincode validates each partial but does not count them at endorsement. The verifier does count them (check 8 of 10) and re-proves every partial, so a sub-threshold tally would be flagged on verification. Endorsement-time counting is identified as the next chaincode revision.	p. 47 to 48, p. 55 (check 8)
You compare against Galal et al., but they ran on different hardware. Is the comparison valid?	The manuscript calls it a cross-study comparison, not a controlled experiment, and says so in Table 3.1 and the Performance section. Workload is matched (single position, four candidates, 50,000 voters, same platform). Hardware is not. The comparison isolates cryptographic design only to the extent hardware is comparable, and the environment is published in Table 3.12 so a reader can judge.	p. 57, Table 3.12
Have you actually run the 3.5 million tier?	The capstone attempts are committed and either outcome is reported as a primary finding. The staged ladder (1, 10, 100, 1,000 validated end to end first) is what makes the attempt disciplined. The scaling-limit rule is stated in advance: below about 98 ballots per second sustained, the result is reported as a limit of the evaluated configuration.	p. 10, p. 38, p. 57 to 58
How do you know the announced tally is right without decrypting ballots?	Homomorphic aggregation: the product of ciphertexts encrypts the sum. Only the aggregate is decrypted, by three of five trustees, each proving honesty. The verifier recomputes the aggregate from the accepted ballots and checks that the announced tally equals it. Accuracy is confirmed against seeded ground truth, $E = 0$.	p. 40 (Algorithm 1), p. 50, p. 55
Does the registrar learn how I voted?	No. Blind signature: the registrar signs a blinded value and never learns the value that later identifies the ballot. The credential on the board is unlinkable to the identity in the registry. The registry is off-chain and private.	p. 41 (Algorithm 4), Table 3.11
What about coercion or vote buying?	Not in the threat model. The study claims unlinkability and end-to-end verifiability, not coercion resistance in the JCJ sense. Say this plainly and point to the adversary classes in Table 3.9.	p. 22 (2.5), Table 3.9

Likely question	Answer from the manuscript	Where
Why is a senatorial race in scope if multi-selection ballots are out?	Single non-transferable vote: each voter still selects exactly one candidate, the leading N fill the seats. The seat count changes how the result is read, not how a ballot is proven, so the existing well-formedness proof (exactly one valid selection) is sufficient. A ballot marking several candidates is what needs a selection-limit proof and stays out.	p. 10, Appendix A
Why five trustees and why three?	The threat model assumes up to two malicious trustees. Three of five means no coalition of two can decrypt and any two can be absent without blocking the count. This is the same quorum shape as ElectionGuard's guardians, whose independence in practice the literature leaves open.	p. 40, Table 3.9, p. 20
What if a trustee is offline during the ceremony?	Any three of the five suffice. Two can be offline. Below three the ceremony fails by design, which is scenario T6.	p. 40, Table 3.8 T6
How is the discrete log recovered at scale?	The decrypted element is g^T with T at most the accepted ballot count. Linear scan at small tiers; baby-step giant-step from 50,000 up. Bound set to accepted ballots so it is exact and terminates.	p. 40
A single accumulator at 3.5 million ballots is a write hotspot. How do you get any throughput?	Per-precinct sub-accumulators keyed by contest and precinct group, folded into the contest total in one closing transaction after voting ends. Correct because ciphertext addition is associative and commutative; the fold is on the board and re-checked by the verifier.	p. 42 (Algorithm 4)
Your synthetic distribution: does it produce a realistic result or ties?	Three profiles. Uniform and skewed alone tie at every scale. The realistic profile applies the skewed rule to most voters and apportions a reserved share down the ranks so every position has a decidable winner and positions differ. Generation is deterministic integer arithmetic, no seed, reproducible across machines.	Appendix A, p. 63 to 64
Is this legal in the Philippines?	No claim of legal validity, compliance, or certification is made. The legal mapping is context for the simulated threats. The election-return section shows the design is compatible with RA 8436/9369 and the 2023 Supreme Court ruling on machine-generated signatures; it strengthens the existing mechanism rather than replacing it.	p. 54 to 55
What is your contribution beyond building a system?	The empirical baseline: integrity, privacy, security, and performance characterized at Philippine scales, which does not currently exist. Plus a reusable domain-agnostic framework, an open verifier, and a benchmark for future work.	p. 11, p. 24 to 25
Why not Ethereum or a public chain?	Public chains carry transaction cost and open participation; the Open Vote Network shows it works only at boardroom scale. Permissioned Fabric gives known participants, governance, throughput, and matches the baseline platform.	p. 21
Where does ElectionGuard fit? Aren't you reimplementing it?	Same primitive family, but ElectionGuard defers the bulletin board to the deployer and leaves guardian independence open. This study fixes the substrate (Fabric), adds blind-signed credentials for cryptographic unlinkability, and evaluates at Philippine scale. Its test vectors are used as the external correctness standard.	p. 20, p. 25, p. 50

Known soft spots (know these before a panelist finds them)

- Chapter III is still written in the future tense. Every experiment is defined, none is reported. If asked for a result, say the evaluation matrix is defined and the recording forms in Appendix C are ready; do not improvise numbers.
- Table 3.1 row 5 points to Chapter IV, which does not exist yet. That is intentional at proposal stage; be ready to say so.
- The on-chain path to Fabric has not yet been executed end to end on the researchers' hardware. This is a development-log fact, not a manuscript claim, so it will only come up if a panelist asks about the demo. Answer: the Fabric run is scheduled on the benchmarking machine in Table 3.12; the offline path is verified.
- The seven properties are named only in Figure 2.3, not enumerated in the body text. Memorize them from Part 2 of this reviewer.
- References [21] to [31] are supporting sources cited in the text but are not in the twenty-study review matrix. If asked why the matrix has twenty rows and the reference list has thirty-one entries, that is the answer.
- The privacy literature citation on p. 51 was corrected to [19] in v3. If you have an older printout it will say [8]; the v3 build is the one to bring.

The goal is not to recite the manuscript. The goal is to show that every claim in it has a defined status, a defined measurement, and a defined limit.

PART 5: THE CODEBASE, SAKSI AND BALOTACHAIN

Everything in this part describes the code as built on 15 September 2026, at `saksi` commit 1812139 and `balotachain` commit a79918d, not the manuscript. Parts 1 to 4 tell you what the thesis promises; this part tells you where each promise lives in the repositories, what changed while building it, and where the code is weaker than the text. Paths are relative to each repository's root.

The two repositories and what each holds

- **Saksi (the framework):** the `saksi` repository. Its packages hold the Rust cryptographic core, the Go chaincode and ledger client, and a Go research console that runs whole elections for the evaluation.
- **BalotaChain (the application):** the `balotachain` repository. It holds the three web apps an operator, a trustee and the public use, and the thesis evidence under `docs`.

The manuscript calls Saksi a Rust library. The cryptographic core is exactly that. The saksi repository around it also carries the chaincode and the console, because the wire format, the on-chain verification and the evaluation harness are versioned together with the core and tested against the same vectors. If a panelist asks, say: the core is the library; the repository is the framework. Terminology still holds: Saksi is the framework, BalotaChain is the application.

Saksi packages (the 11 you may be asked to open)

Package	Language	What it holds	Manuscript component
<code>packages/saksi-crypto</code>	Rust	<code>ristretto255</code> ElGamal, CDS and Chaum-Pedersen proofs, Schnorr signatures, Pedersen DKG	Saksi cryptographic core
<code>packages/saksi-credentials</code>	Rust	blind-signed credentials, presentations, position nullifiers	Registration (stage 2)
<code>packages/saksi-protocol</code>	protobuf, Rust, Go	the wire format (<code>wire.proto</code>) and golden test vectors shared by Rust and Go	Published test vectors
<code>packages/saksi-auditor</code>	Rust	the synthetic population generator, the streaming verifier, ground-truth tables	Open-source verifier; data collection
<code>packages/saksi-demo</code>	Rust	the command line: <code>gen</code> , <code>gen-ground-truth</code> , <code>audit</code> , <code>audit-stream</code>	Verifier entry point
<code>packages/saksi-bulletin/chaincode</code>	Go	14 contract functions, plus <code>cdsverify</code> , <code>credverify</code> and <code>sigverify</code>	Fabric chaincode
<code>packages/saksi-bulletin/client-sdk</code>	Go	gateway connection, ledger receipts, chain walk, the load driver (<code>bench</code>)	Blockchain integration
<code>packages/saksi-bulletin/network</code>	shell, YAML	the Fabric test network and its declared orderer batch parameters	Table 3.12 environment
<code>packages/saksi-campaign</code>	Go, HTML	the research console, its wizard, campaigns, the attack timeline	Evaluation tool (Appendix C)
<code>packages/saksi-ffi-flutter</code> , <code>saksi-ffi-tauri</code>	Rust	bindings for mobile and desktop clients	Flutter and Tauri clients; not on the evaluated path
<code>tools</code>	shell	<code>up.sh</code> , <code>tier.sh</code> , <code>ladder.sh</code> , <code>t3-restart.sh</code>	Deployment steps; validation ladder

BalotaChain directories: what is live and what is legacy

Directory	Status	What it holds
apps/admin	Live	Election admin web app: create an election, check the population, run it, open the ceremony, publish, verify
apps/trustee	Live	Trustee web app: sign in, submit your own shares, publish once the threshold is met
apps/auditor	Live	Public bulletin board: results after publication, integrity checks, verify a tracking code
packages/ui	Live	Shared design system (@balotachain/ui) used by the three apps
docs	Live	CLAIMS.md, manuscript-amendments.md, desktop-runs evidence, the cost model
apps/voter	Legacy	Flutter screens from the one-voter demo; not connected to the console
apps/*/src-tauri	Legacy	Tauri desktop shells over a file-backed stand-in; no longer built
crates/*, services/fabric-adapter	Legacy	The path from before the console: file store, HTTP gateway, one-voter runner, read-only chain adapter

The three live apps are built into one folder and served by the saksi console at /admin/, /trustee/ and /board/, on the same origin as its API. That is why they have no backend of their own, and why the console's origin checks and session cookie protect them unchanged.

Figure 3.3's 7 stages, located in code

Stage	What the code does	Where
1. Setup	The generator builds the election parameters and the DKG transcript; the chaincode records them (CreateElection, PublishDKGTranscript).	saksi-auditor/src/fixtures.rs (gen_prologue); saksi-bulletin/chaincode/contract.go
2. Registration	One issuer keypair per run; one blind-signed credential per synthetic voter.	fixtures.rs (build_voter); saksi-credentials/src/issuance.rs
3. Voting	One ciphertext per candidate, each with a CDS proof, plus a per-position nullifier.	fixtures.rs (build_voter); saksi-crypto/src/nizk/cds.rs
4. Submission	Ballots go through a timed window. The chaincode checks, in order: decode, shape, credential signature, election exists and is open, nullifier unspent, CDS proof.	saksi-campaign/executor.go (submitBallots); chaincode/contract.go (SubmitBallot)
5. Aggregation	The verifier sums eligible ballots per contest as it streams them. Nothing is accumulated on-chain.	saksi-auditor/src/lib.rs (audit_streaming_chunked)
6. Tally ceremony	Trustees submit shares; the console refuses to publish below the threshold; the chaincode refuses a tally without threshold-many valid trustee signatures.	saksi-campaign/ceremony.go; chaincode/contract.go (PublishTally); chaincode/sigverify
7. Verification	audit-stream re-derives the result from the published files, and for on-chain runs from the chain's own copy as well.	saksi-auditor/src/demo.rs; saksi-campaign/ledger_dump.go

Stage 5 is where code and manuscript differ most. Algorithm 4's per-precinct sub-accumulators were not built: with no on-chain accumulator there is no hot key to contend for, and the verifier recomputes every sum from the ballots. The change is written up in docs/manuscript-amendments.md; know that it exists before you defend Algorithm 4.

The verifier's 10 checks (Table 3.11), located in code

Check	Verifier check id (saksi-auditor)	Also enforced on-chain?
(1) every recorded transaction is valid	ballot.decode, ballot.shape	Yes, decode and shape gates at endorsement
(2) each ballot's credential is valid	ballot.credential, ballot.issuer_binding	Yes, credential gate
(3) every nullifier is unique	nullifier.unique	Yes, nullifier gate
(4) every ciphertext is well-formed	ballot.shape	Yes, shape gate
(5) every ballot's disjunctive proof	ballot.cds_proof	Yes, cds gate
(6) the homomorphic aggregate is correct	tally.aggregate	No
(7) each trustee's Chaum-Pedersen proof	decryption.cp_proof	Presence only (cp-presence)
(8) at least three of five trustees contributed	decryption.threshold, tally.signatures	Yes, the PublishTally signature gate
(9) the announced tally equals the decrypted aggregate	tally.homomorphic_sum	No
(10) append-only consistency of the ledger	VerifyChain in client-sdk/ledger.go, run by the console's verify-only step	Not a chaincode check

Two more checks exist that the manuscript does not list: stream.completeness (as many ballots audited as the header declares) and tally.accuracy ($E = 0$ against the seeded ground truth). Every check has a stable id, and attack verdicts are scored against those ids.

The research console behind Chapter IV

- **Single election:** through the console's wizard or the admin app. Generate, check the data, encrypt and record, trustee ceremony, verify, then attack it.
- **Measurement campaign:** warm-ups and measured repetitions of one configuration, summarised in summary.csv. Attacks are always off, because a security run perturbs timing.
- **Gates before a run:** the validation ladder (1, 10, 100 and 1,000 voters) must have passed on the same build before any larger tier; the console also refuses a tier that will not fit the disk or memory, or whose longest phase would outlast the phase timeout.
- **Attack timeline:** the lifecycle pauses before ballots, part-way through submission, after the close and during the ceremony. A PASS counts only when the attack was refused by the gate it targets; a refusal by any other gate is INCONCLUSIVE.
- **Resilience (T3):** a fault stops the peer part-way through submission; resume submits only what the chain lacks and closes the election; verify-only reconciles the chain against the run.
- **Evidence per run:** journal.ndjson (line 1 records the environment and commits), perf.csv, correctness.csv, negative-tests.csv, and a campaign export zip for the cost model.

This is your answer on traceability (Software development, pp. 42 to 44): every run folder names the saksi commit in run.json and in journal line 1, and a failed repetition is recorded with its reason rather than dropped.

The browser apps and the console API

App	Calls	Who may call
Admin	POST /generate, GET /api/check, POST /ceremony/start, POST /verify	Admin session
Trustee	POST /ceremony/submit (own shares only), POST /ceremony/publish	Trustee session; publish by any signed-in user
Public board	GET /api/board, GET /api/verify-code	Anyone

Sign-in is demo-grade and you should say so: users in a file with bcrypt passwords, sessions in memory, no TLS termination, the console bound to localhost by default. A trustee's identity comes from the session, never from the link, so a trustee cannot submit another trustee's shares.

Easy-to-confuse pairs: manuscript versus code

The manuscript says	The code does
Flutter voter application; Tauri trustee console	Browser apps served by the console (admin, trustee, board). The Flutter and Tauri code is legacy. No person casts a ballot on the evaluated path; the generator builds every voter, as Chapter III's synthetic data design requires.
Private keys and nonces are uniformly random from the OS CSPRNG (Algorithms, pp. 39 to 42)	True, including every trustee's DKG polynomial since saksi #50 (commit 1812139), which replaced the generator's fixed dealer coefficients with Dealer::random. Runs generated before that commit used the fixed coefficients.
Chaincode with three gates	Named gates. A ballot passes decode, shape, credential, election-exists, election-open, nullifier, dkg-missing and cds; DKG transcripts, partials and tallies have their own.
Per-precinct sub-accumulators (Algorithm 4)	Not built. Aggregation happens in the verifier, so there is no hot key.
Threshold enforced by the console, not the chaincode (this reviewer's Panel Q&A)	Changed. The chaincode now refuses a tally unless every signature verifies and at least threshold distinct trustees signed. Chaum-Pedersen proofs are still only checked for presence on-chain.
The on-chain path has not been executed end to end (this reviewer's Known soft spots)	No longer true. It has been run on the desktop Fabric network, and the runs are recorded in balotachain docs/desktop-runs.
Five trustees, three of five	The wizard's default. The console accepts any t of n up to 15 trustees.

Panel Q&A: questions about the code

Likely question	Answer from the code	Where
Which commit produced your results?	Every run records the saksi commit in run.json and journal line 1, and the ladder that unlocks large tiers is tied to one build.	saksi-campaign/env.go, runstore.go
Does the chaincode verify proofs, or trust the client?	It verifies the CDS proof and the issuer's signature on the credential at endorsement, and refuses with a named gate. It does not verify partial-decryption proofs; the verifier does.	chaincode/contract.go (SubmitBallot); chaincode/cdsverify

Likely question	Answer from the code	Where
Is the verifier independent of the code that made the election?	It is a separate program that reads only published files, and for on-chain runs it also audits the chain's own copy. It shares Saksi's Rust crates with the generator, so the Go chaincode's proof check is the independent second implementation for ballots.	saksi-auditor/src/demo.rs; saksi-campaign/ledger_dump.go
How do you know an attack was stopped by the right check?	Every refusal carries a gate id and a PASS needs the declared one; anything else is INCONCLUSIVE. Earlier on-chain verdicts scored without this were discarded, not reused.	saksi-campaign/scenarios.go (classifyLive, classifyAudit)
Could someone decrypt your generated elections without the trustees?	Not from runs at saksi commit 1812139 or later: every dealer polynomial is random. Older runs used fixed dealers and are cited only for correctness, verifiability and performance. What remains is the simulated ceremony: one process holds every share.	saksi-crypto/src/dkg.rs (Dealer::random); saksi-auditor/src/fixtures.rs (gen_prologue)
Who can submit to the chaincode?	Any identity on the channel; no function checks the caller. Every submission still passes the gates, so the risk is denial of service (claiming an election id first), not forgery.	chaincode/contract.go
What happens if a peer fails mid-election?	The T3 fault stops the peer during submission. Resume submits only the ballots the chain does not hold, then closes the election, and verify-only reconciles the chain against the run.	saksi-campaign/fault.go; executor.go (resumeAndClose)
Why is ballot ordering not protected?	The tally does not depend on order, and no check looks at it. Reordering is recorded as a SKIPPED attack instead of being claimed as detected.	saksi-campaign/scenarios.go (Registry)
How does a voter check their ballot?	On the public board, with a tracking code made from the first 8 hex digits of the ballot's nullifier.	saksi-campaign/board.go (handleVerifyCode)

Known soft spots in the code (know these before a panelist finds them)

- **Older runs used fixed DKG dealers.** Runs made before saksi commit 1812139 have keys derivable from the source: cite them for correctness, verifiability and performance only (each run.json names its commit).
- **The ceremony is simulated.** The generator produces every partial decryption and signature in one process, and the console forwards them when a trustee acts, so that process holds every key share. The cryptography is real; separation between trustees is not exercised.
- **No caller authorization in the chaincode.** Anyone on the channel can call any function; the gates still apply to what they submit.
- **Ordering is not detected** on-chain or by the verifier.
- **Chaum-Pedersen proofs are checked for presence on-chain.** A share with a wrong proof is recorded and caught only by the verifier.
- **One machine.** The test network's two organisations and single orderer run on one computer, so distributed trust between operators is not demonstrated.
- **No live voter.** The Flutter app is not connected; the Tauri shells and the one-voter runner in crates are left over from the earlier demo and are not built by CI.
- **An open gap on the public board.** Its download links still point at the admin-only export route, although the console already serves a public copy of the verification files.

The code is the strongest evidence you have. For every part of it, know whether it is implemented, demonstrated, or a stated limit, and say which before you are asked.